AIR UNIVERSITY

Department of Computer Science

COAL SEMESTER PROJECT

Computer Organization & Assembly Language

Real-Time Digital Clock

&

Stopwatch System

with 7-Segment ASCII Display & Multi-Mode UI

Developer Wasif Ali	**Instructor** Sir Imran
Subject COAL (CS-221)	**Institution** Air University

Table of Contents

1. Executive Summary 4
2. Introduction 5
2.1 Project Background 5
2.2 Problem Statement 5
2.3 Scope 5
2.4 Tools and Environment 6
3. System Architecture 7
3.1 High-Level Architecture Overview 7
3.2 State Machine Design 8
3.3 Main Application Loop 9
3.4 Screen Coordinate System 9
4. Data Segment Reference 10
4.1 System Time Structure 10
4.2 Visual / Rendering Variables 10
4.3 State Machine Variables 11
4.4 Stopwatch Variables 11
4.5 UI String Table 11
5. Procedure Reference 12
5.1 main PROC 12
5.2 DrawUIHeader PROC 12
5.3 HandleInput PROC 12
5.4 WriteZeroPadded PROC 13
5.5 DrawHugeDigit PROC 13
5.6 Segment Procedures (SegA – SegG) 14
5.7 DrawColon PROC 15
5.8 ClearDigitArea PROC 15
6. Mode Implementation Details 16
6.1 Clock Mode (Mode 0) 16
6.1.1 Wine Environment Offset 16
6.1.2 12-Hour AM/PM Conversion 16
6.1.3 Date Rendering 16
6.2 Stopwatch Mode (Mode 1) 17
6.2.1 Time Calculation 17
6.2.2 Time Decomposition 18
6.3 About Mode (Mode 2) 18
7. The 7-Segment Rendering Engine 19
7.1 Design Philosophy 19
7.2 Segment Coordinate Map 19
7.3 The blockChar Variable 20
7.4 Color Animation 20

8. Memory and Register Usage 21
8.1 Register Conventions 21
8.2 Stack Usage 21
8.3 Data Segment Memory Map 22
9. Windows API Integration 23
9.1 GetLocalTime 23
9.2 GetMSeconds (Irvine32 Wrapper) 23
9.3 Irvine32 Procedures Used 23
10. User Interface Design 25
10.1 Screen Layout Overview 25
10.2 Mode-Specific UI Elements 26
10.2.1 Clock Mode 26
10.2.2 Stopwatch Mode 26
10.2.3 About Mode 26
10.3 Footer Key Hints 26
10.4 Keyboard Controls Summary 27
11. Key Algorithms 28
11.1 Two-Digit Decimal Extraction 28
11.2 Non-Blocking Input with Polling 28
11.3 Stopwatch Pause-Resume Algorithm 28
12. Known Issues and Design Decisions 30
12.1 Wine Environment Time Offset 30
12.2 32-Bit Timer Wrap-Around 30
12.3 Color Cycling Scope 30
12.4 Fixed Console Size Assumption 30
12.5 Single-Character blockChar 30
13. Testing and Validation 31
13.1 Test Scenarios 31
13.2 Screenshots 31
14. COAL Concepts Demonstrated 33
14.1 Course Learning Outcomes 33
14.2 Advanced Techniques 33
15. Conclusion 34
15.1 Summary 34
15.2 Challenges Overcome 34
15.3 Future Enhancements 34
Appendix A: Complete Source Code Listing 35
Appendix B: Irvine32 Color Constants Reference 36
Appendix C: Project File Structure 37
Build Command Reference 37
Appendix D: Glossary 38

1. Executive Summary

This document presents the complete technical documentation for the Real-Time Digital Clock and Stopwatch System, developed as a semester project for the Computer Organization and Assembly Language (COAL) course at Air University. The project was independently designed and implemented by Wasif Ali under the supervision of Instructor Sir Imran.

The system is a feature-rich, console-based application written entirely in x86 MASM (Microsoft Macro Assembler) using the Irvine32 library. It demonstrates mastery of low-level programming concepts including direct memory manipulation, system time retrieval via Windows API calls, interrupt-driven input handling, hardware register management, and complex rendering logic all implemented without the assistance of any high-level language constructs.

The application provides three distinct operational modes: A Real-Time Clock with date display, a precision Stopwatch with millisecond resolution, and an About/Credits screen. The display engine renders oversized digits using a custom-built 7-segment ASCII rendering system, producing a visually striking and professional user interface directly within a Windows console window.

Key Highlights	
Platform:	x86 Windows (MASM + Irvine32 library)
Language:	Intel x86-32 Assembly
Lines of Code:	~550+ assembly instructions
Display Engine:	Custom 7-Segment ASCII renderer
Modes:	Clock, Stopwatch, About
Input:	Non-blocking keyboard polling
Time Source:	Windows GetLocalTime API

2. Introduction

2.1 Project Background

Assembly language programming sits at the foundation of computer science education. Unlike high-level languages that abstract away hardware details, assembly demands a deep, intimate understanding of processor registers, memory addressing modes, arithmetic flags, the call stack, and I/O mechanisms. The COAL Semester Project was assigned to challenge students to apply this knowledge in a practical, real-world context.

The Real-Time Digital Clock and Stopwatch is a non-trivial system that requires the programmer to simultaneously manage real-time system calls, user interface rendering, state machine logic, and keyboard I/O all while respecting the strict constraints of register availability and memory layout inherent to assembly programming.

2.2 Problem Statement

Design and implement a multi-mode time display application in x86 Assembly that:

- Retrieves and displays the live system time with second-by-second accuracy.
- Renders time digits in an enlarged, readable 7-segment font constructed entirely from ASCII block characters.
- Provides a fully functional stopwatch with start, stop, and reset functionality at centisecond precision.
- Supports dynamic switching between 12-hour AM/PM and 24-hour display formats.
- Implements a clean, navigable multi-screen UI entirely within a Windows console.
- Handles all user input asynchronously without blocking the display update loop.

2.3 Scope

This project covers the complete design and implementation lifecycle, from architectural decisions and data structure design to low-level procedure implementation and system integration. The documentation covers:

- System architecture and state machine design.
- Memory layout and all data segment variables.
- Detailed explanation of every procedure and subroutine.
- The 7-segment rendering algorithm and coordinate system.
- Time management, including Windows API integration.
- Input handling and mode-switching logic.
- Known issues, constraints, and design trade-offs.

2.4 Tools and Environment

Assembler	MASM (Microsoft Macro Assembler) via Visual Studio / MASM32 SDK
Library	Irvine32.lib — Kip Irvine's x86 helper library
Include File	Irvine32.inc — macro definitions, constants, prototypes
Target OS	Windows (Win32 subsystem, 32-bit mode)
Compatibility	Also tested under Wine (Linux compatibility layer)
IDE	Visual Studio / MASM32 command-line toolchain
Console	Windows CMD or compatible terminal emulator

3. System Architecture

3.1 High-Level Architecture Overview

The application follows a single-threaded, polling-based event loop architecture common in real-time embedded systems. A continuous main loop queries the current time, checks for keyboard input, and delegates rendering to the appropriate mode handler. This design avoids multi-threading complexity while still achieving smooth real-time updates at a 50ms refresh rate (20 frames per second).

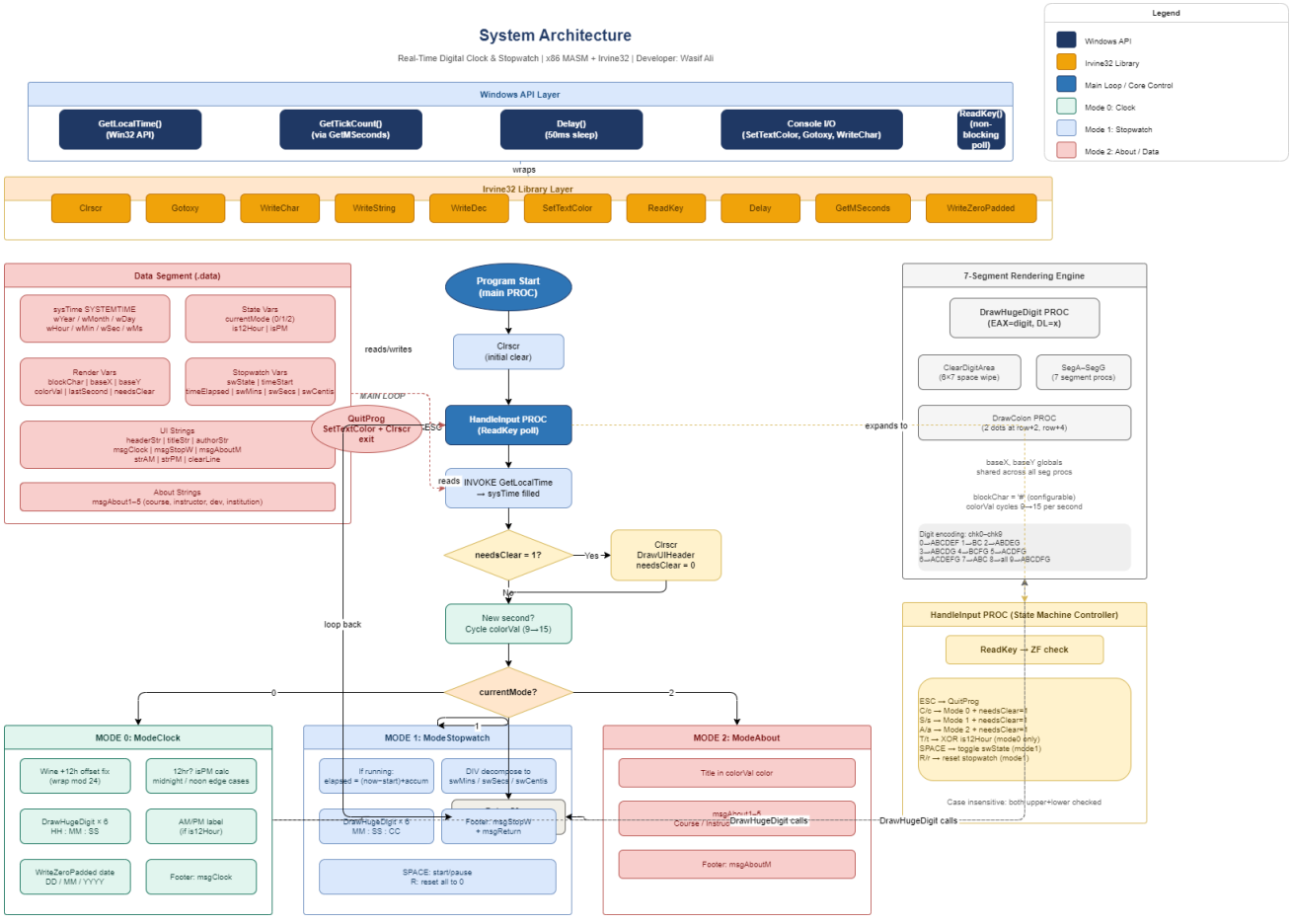


Figure: System Architecture Diagram — Main Loop & Component Interaction

3.2 State Machine Design

The application's core logic is driven by a finite state machine (FSM) with three states. The currentMode variable (stored in the data segment as a single byte) holds the active state at all times. Transitions are triggered by specific keyboard inputs processed in the HandleInput procedure.

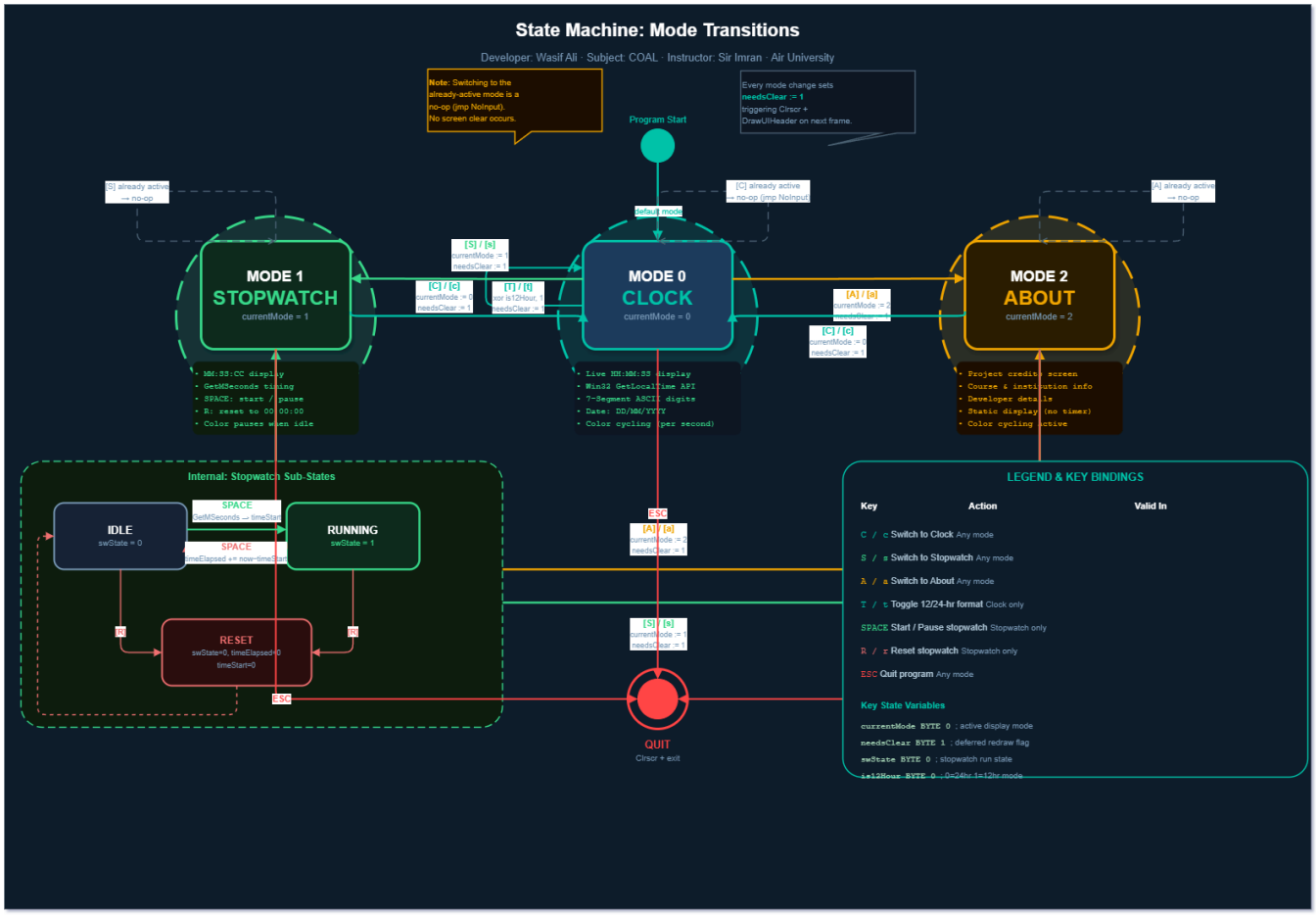


Figure: State Machine Diagram — Mode Transitions (Clock / Stopwatch / About)

State Value	Mode Name	Entry Key	Description
0	Clock	[C]	Displays live hours, minutes, seconds, and date
1	Stopwatch	[S]	Precision stopwatch: MM:SS:CC format
2	About	[A]	Credits screen with project and developer info

3.3 Main Application Loop

The MainLoop label at the top of main PROC forms the heartbeat of the application. Every iteration performs the following steps in sequence:

1. Call HandleInput to consume any pending keypress.
2. Call GetLocalTime to refresh the sysTime SYSTEMTIME structure.
3. If needsClear = 1, clear the screen, redraw the static UI header, and reset the flag.
4. If a new second has elapsed (lastSecond differs from sysTime.wSecond), cycle the color and update lastSecond.
5. Branch to the appropriate mode renderer (ModeClock, ModeStopwatch, or ModeAbout).
6. After rendering, call Delay 50ms, then jump back to MainLoop.

The 50ms delay gives a refresh rate of approximately 20 Hz, which is smooth enough for clock display and provides centisecond-resolution updates in Stopwatch mode without consuming 100% CPU.

3.4 Screen Coordinate System

The Irvine32 library uses a Gotoxy(column, row) coordinate system where (0, 0) is the top-left corner of the console. Columns increase left-to-right (the DL register), and rows increase top-to-bottom (the DH register). All rendering functions in this project use this coordinate system.

UI Region	DL (Column)	DH (Row)	Content
Header	15	1 – 4	Title, author, separator lines
Digit Display	10–76	8 – 15	7-Segment large digits
AM/PM Label	77	13	AM / PM text (12-hour mode only)
Date Display	33	18	DD / MM / YYYY
Footer Row 1	12	21	Mode-specific key hints
Footer Row 2	12	22	Secondary key hints

4. Data Segment Reference

4.1 System Time Structure

The Windows SYSTEMTIME structure is used for all date and time retrieval. It is defined in the Irvine32 include files and populated by calling the Win32 GetLocalTime API function:

```
sysTime    SYSTEMTIME <>

; SYSTEMTIME structure members (all WORD = 16-bit):
;  wYear         — 4-digit year (e.g., 2024)
;  wMonth        — Month: 1 (January) to 12 (December)
;  wDayOfWeek    — Day of week: 0 (Sunday) to 6 (Saturday)
;  wDay          — Day of month: 1 to 31
;  wHour         — Hour: 0 to 23 (always 24-hour from API)
;  wMinute       — Minute: 0 to 59
;  wSecond       — Second: 0 to 59
;  wMilliseconds — Milliseconds: 0 to 999
```

4.2 Visual / Rendering Variables

Variable	Type	Initial Value	Purpose
blockChar	BYTE	'#'	ASCII character used to draw all 7-segment blocks
baseX	BYTE	?	X-coordinate (column) of the current digit being drawn
baseY	BYTE	8	Y-coordinate (row) of the top of all digit segments
colorVal	BYTE	10	Current foreground color index (1–15, Irvine palette)
lastSecond	BYTE	255	Tracks last-seen second value to detect time changes
needsClear	BYTE	1	When set to 1, triggers a full screen clear and redraw

The colorVal variable cycles through Irvine32 color constants (9 through 15) once per second, producing a rainbow color animation on the displayed digits. The cycle is paused when the Stopwatch is in the stopped state.

4.3 State Machine Variables

Variable	Type	Values	Meaning
currentMode	BYTE	0, 1, 2	Active screen: 0=Clock, 1=Stopwatch, 2=About
is12Hour	BYTE	0 or 1	24-hour (0) vs 12-hour AM/PM (1) display toggle
isPM	BYTE	0 or 1	Computed each frame: 0=AM, 1=PM (12-hour mode only)

4.4 Stopwatch Variables

Variable	Type	Description
swState	BYTE	0 = Stopped, 1 = Running
timeStart	DWORD	GetTickCount ms value at the moment the stopwatch was started
timeElapsed	DWORD	Accumulated elapsed time in milliseconds across pause/resume cycles
swMins	DWORD	Computed display value: minutes (0–99)
swSecs	DWORD	Computed display value: seconds (0–59)
swCentis	DWORD	Computed display value: centiseconds (0–99)

The Irvine32 library provides a GetMSeconds procedure that wraps the Windows GetTickCount system call, returning a 32-bit count of milliseconds since system boot. By recording the start timestamp and subtracting it from the current timestamp on each frame, elapsed time is computed with high accuracy. Pausing accumulates the elapsed value into timeElapsed before storing zero in timeStart, allowing seamless resume from where the timer left off.

4.5 UI String Table

Label	Content Summary
headerStr	49-character '=' separator line
titleStr	Centered project title banner text
authorStr	Subject and developer credit line
slashStr	" / " — date separator literal
strAM	" AM " — displayed in 12-hour AM mode
strPM	" PM " — displayed in 12-hour PM mode
msgClock	Footer hint string for Clock mode
msgStopW	Footer hint string for Stopwatch mode
msgAboutM	Footer hint string for About mode
msgReturn	Secondary footer: [C] Return to Clock
clearLine	50-space string used to erase footer lines during mode transitions
msgAbout1–5	About screen content lines: title, course, instructor, developer, institution

5. Procedure Reference

5.1 main PROC

The entry point of the application. Responsibilities:

- Calls Clrscr to clear the console on startup.
- Jumps into the MainLoop, which never exits except via the ESC key path leading to QuitProg.
- Manages the top-level conditional branching to ModeClock, ModeStopwatch, and ModeAbout.

The QuitProg label (exposed with double colon :: for global visibility) resets the text color to light gray, clears the screen, and terminates the program via the Irvine32 exit macro.

5.2 DrawUIHeader PROC

Draws the static decorative header that appears at the top of the screen. This procedure is only called when the screen has been freshly cleared (i.e., after a mode transition or on startup). It renders four lines:

- Row 1: '=' separator line
- Row 2: Centered project title
- Row 3: Subject and developer credits
- Row 4: Closing '=' separator line

All text is rendered in white on black to create a distinct banner area that contrasts with the animated colored digits below.

```
DrawUIHeader PROC
    mov  eax, white + (black * 16)  ; Set white text on black background
    call SetTextColor
    mov  dl, 15  |  mov  dh, 1
    call Gotoxy
    mov  edx, OFFSET headerStr
    call WriteString
    ; ... repeated for title, author, footer separator
    ret
DrawUIHeader ENDP
```

5.3 HandleInput PROC

The non-blocking input handler. It calls Irvine32's ReadKey procedure, which returns immediately with ZF=1 if no key was pressed, or with the key's ASCII code in AL if a key is available. This design ensures the main loop is never blocked waiting for user input.

The procedure implements the full state machine transition logic:

Key(s)	Condition	Action
ESC (27)	Any mode	Jump to QuitProg — exit application
C / c	Any mode	Set currentMode=0, needsClear=1
S / s	Any mode	Set currentMode=1, needsClear=1
A / a	Any mode	Set currentMode=2, needsClear=1
T / t	Mode=0 only	XOR is12Hour — toggle 12/24hr; set needsClear=1
SPACE	Mode=1 only	Toggle swState; record timeStart or accumulate timeElapsed
R / r	Mode=1 only	Reset swState, timeElapsed, timeStart to 0

Case-insensitive handling is achieved by checking both uppercase and lowercase ASCII values for each key. The XOR instruction provides an elegant single-instruction toggle for binary state variables (is12Hour, swState).

5.4 WriteZeroPadded PROC

A utility procedure that prints a number (in EAX) with a leading zero if it is less than 10. This ensures the date is always displayed as DD/MM/YYYY rather than D/M/YYYY.

```
WriteZeroPadded PROC
    cmp  eax, 10
    jae  PrintNumber       ; If >= 10, no padding needed
    push eax
    mov  al, '0'
    call WriteChar          ; Print leading zero
    pop  eax
PrintNumber:
    call WriteDec           ; Print the number itself
    ret
WriteZeroPadded ENDP
```

5.5 DrawHugeDigit PROC

The centerpiece rendering procedure. It receives the digit value (0–9) in EAX and the X-coordinate of the digit's left edge in DL. It first calls ClearDigitArea to erase the previous frame's digit, then calls the appropriate combination of segment-drawing sub-procedures (SegA through SegG) based on which segments are active for that digit.

The 7-segment encoding table is implemented as a chain of compare-and-jump instructions:

Digit	Active Segments	Inactive Segments
0	A, B, C, D, E, F	G

1	B, C	A, D, E, F, G
2	A, B, D, E, G	C, F
3	A, B, C, D, G	E, F
4	B, C, F, G	A, D, E
5	A, C, D, F, G	B, E
6	A, C, D, E, F, G	B
7	A, B, C	D, E, F, G
8	A, B, C, D, E, F, G	— (all on)
9	A, B, C, D, F, G	E

5.6 Segment Procedures (SegA – SegG)

Each of the seven segment procedures draws one segment of the 7-segment display at the coordinates derived from baseX and baseY. The segment layout follows the standard 7-segment convention:

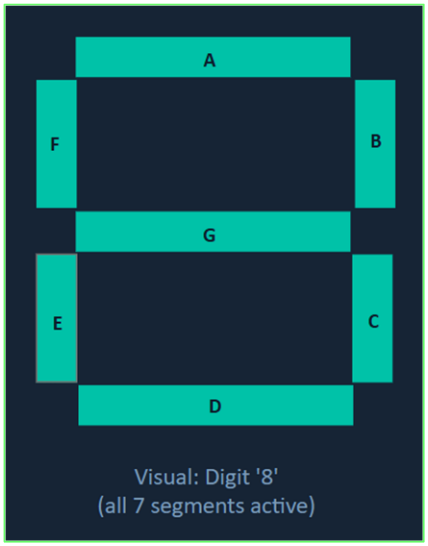


Figure: 7-Segment Layout Diagram showing positions of Segments A–G

Procedure	Position	Characters Written	Rows Used
SegA	Top horizontal bar	4 block chars	baseY
SegB	Top-right vertical stroke	1 char × 2	baseY+1, baseY+2
SegC	Bottom-right vertical stroke	1 char × 2	baseY+4, baseY+5
SegD	Bottom horizontal bar	4 block chars	baseY+6
SegE	Bottom-left vertical stroke	1 char × 2	baseY+4, baseY+5
SegF	Top-left vertical stroke	1 char × 2	baseY+1, baseY+2
SegG	Middle horizontal bar	4 block chars	baseY+3

All procedures use the global baseX variable (set by DrawHugeDigit) as the anchor point, adding fixed offsets to position each segment precisely. Vertical strokes are drawn with individual Gotoxy + WriteChar calls on consecutive rows. Horizontal bars use four consecutive WriteChar calls on a single row.

5.7 DrawColon PROC

Renders a colon separator between digit pairs (HH:MM:SS). It draws a single blockChar at DH = baseY+2 and another at DH = baseY+4, creating the classic two-dot colon of a digital clock. The X-coordinate is passed in DL and the procedure does not modify baseX.

5.8 ClearDigitArea PROC

Before each digit is drawn, this procedure erases the 6-column by 7-row rectangular area that the digit occupies. It loops over 7 rows (ECX = 7), positioning the cursor at (baseX, row) and writing six space characters on each row. This prevents ghosting when digits change (e.g., when '8' changes to '1', the unused segments must be cleared).

```
ClearDigitArea PROC
    push eax  |  push ecx
    mov  dh, baseY
    mov  ecx, 7              ; Loop over 7 rows
clrLoop:
    mov  dl, baseX
    call Gotoxy
    mov  al, ' '
    call WriteChar            ; 6 spaces to erase the digit cell
    call WriteChar
    call WriteChar
    call WriteChar
    call WriteChar
    call WriteChar
    inc  dh
    loop clrLoop
    pop  ecx  |  pop  eax
    ret
ClearDigitArea ENDP
```

6. Mode Implementation Details

6.1 Clock Mode (Mode 0)

Clock mode retrieves the current hour from sysTime.wHour and applies two sequential transformations before rendering:

6.1.1 Wine Environment Offset

The developer noted that under Wine (a Windows compatibility layer for Linux), GetLocalTime consistently returns a value 12 hours behind the actual local time. To correct for this, the code adds 12 to wHour and wraps with modulo-24 arithmetic:

```
movzx eax, sysTime.wHour
add   eax, 12          ; Wine offset fix
cmp   eax, 24
jb    TimeFixed
sub   eax, 24          ; Wrap around midnight
TimeFixed:
```

6.1.2 12-Hour AM/PM Conversion

If is12Hour = 1, the corrected 24-hour value is converted to 12-hour format using bulletproof edge-case handling:

24-Hour Input	Condition	12-Hour Output	AM/PM
0 (midnight)	= 0	12	AM
1 – 11	< 12	unchanged	AM
12 (noon)	= 12	12	PM
13 – 23	> 12	subtract 12	PM

After conversion, digit pairs are separated using integer division (DIV BL where BL=10) to extract the tens digit (AL) and units digit (AH). Each digit is drawn at the correct X offset using DrawHugeDigit, with DrawColon called between each digit pair. The push/pop pattern around DIV preserves both digits across the two DrawHugeDigit calls.

6.1.3 Date Rendering

The date is printed on row 18 in white text using WriteZeroPadded for day and month, and WriteDec for the four-digit year. The slashStr literal provides the '/' separator characters between fields.

6.2 Stopwatch Mode (Mode 1)

The stopwatch uses Windows millisecond tick counts to measure elapsed time with high precision. The timing model is designed to correctly handle multiple pause/resume cycles:

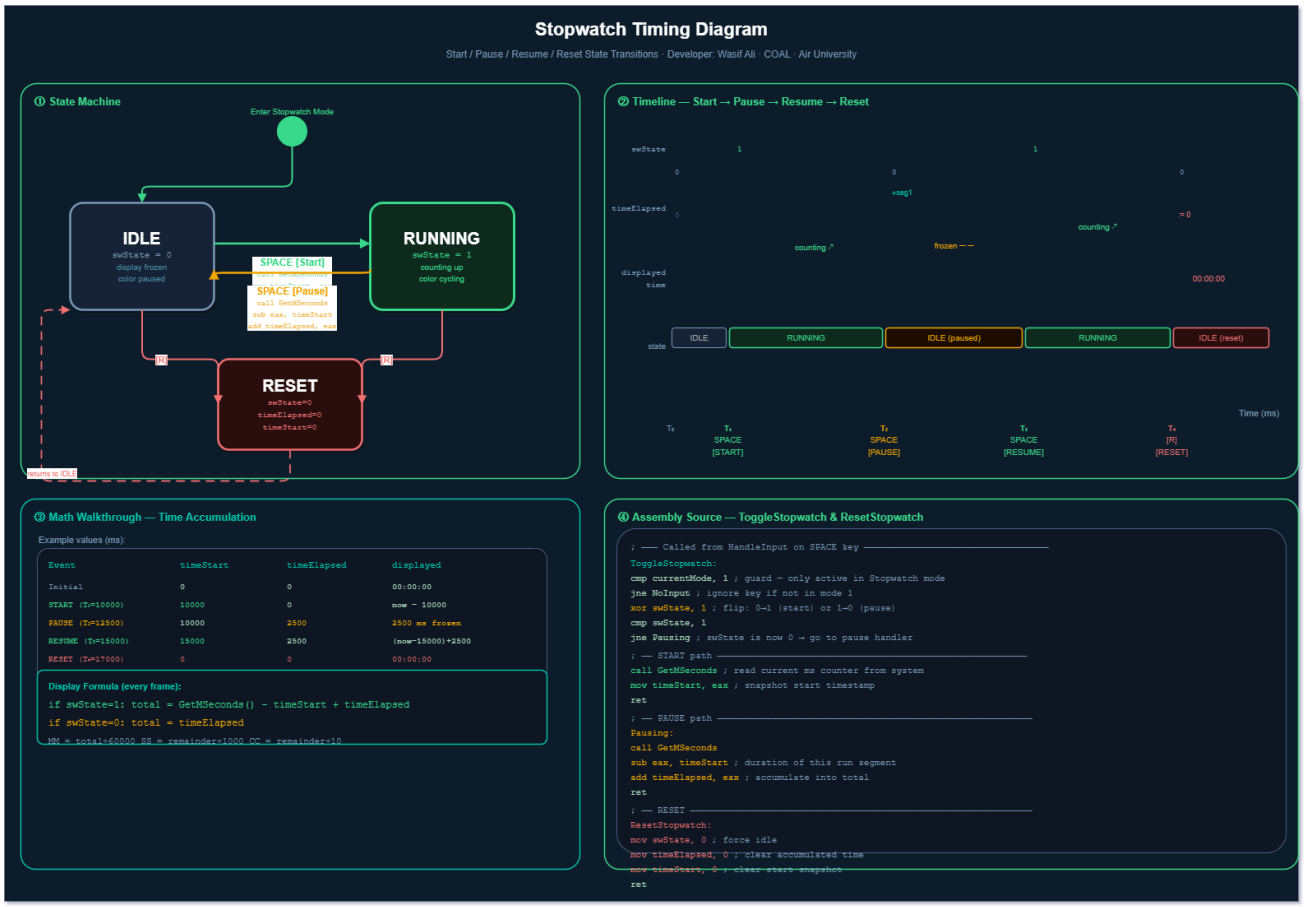


Figure: Stopwatch Timing Diagram — Start/Pause/Resume/Reset state transitions

6.2.1 Time Calculation

On each frame when swState = 1 (running), elapsed time is calculated as:

```
; Elapsed = (currentTick - timeStart) + timeElapsed
call   GetMSeconds          ; EAX = current millisecond tick
sub    eax, timeStart       ; EAX = ms since last Start pressed
add    eax, timeElapsed     ; Add any previously accumulated time
```

When the stopwatch is paused, the elapsed duration is accumulated into timeElapsed so it is preserved across future start/stop cycles. On Reset, all three variables are zeroed.

6.2.2 Time Decomposition

The raw millisecond value is decomposed into display components using successive DIV operations:

```
; Total ms -> Minutes
mov  edx, 0   |  mov  ebx, 60000
div  ebx                        ; EAX = minutes, EDX = remainder ms
mov  swMins, eax

; Remainder -> Seconds
mov  eax, edx  |  mov  edx, 0  |  mov  ebx, 1000
div  ebx                        ; EAX = seconds, EDX = remainder ms
mov  swSecs, eax

; Remainder -> Centiseconds (0-99)
mov  eax, edx  |  mov  edx, 0  |  mov  ebx, 10
div  ebx                        ; EAX = centiseconds
mov  swCentis, eax
```

6.3 About Mode (Mode 2)

The About screen is a static display rendered in two text styles: the project title in the current animated color (colorVal), and the remaining detail lines in white on black. Five lines cover the course name, instructor, developer, and institution. The footer provides navigation back to Clock and Stopwatch modes.

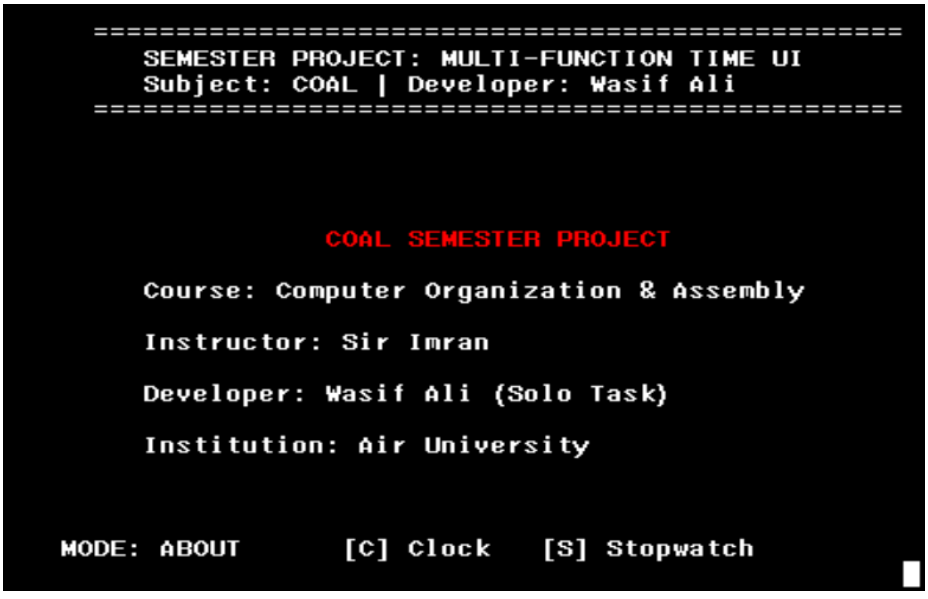


Figure: About / Credits Screen

7. The 7-Segment Rendering Engine

7.1 Design Philosophy

Rendering large, visually impactful digits in a text-only console is a classic challenge in terminal programming. This project solves it by replicating the structure of a physical 7-segment LED display — the type used in calculators, clocks, and scoreboards — using ASCII '#' characters placed at precisely calculated console coordinates.

Each digit occupies a cell that is 6 columns wide and 7 rows tall. Digits are placed at fixed X offsets: 10, 20, 35, 45, 60, 70 (with colons at columns 30 and 55). All digits share the same Y baseline (baseY = 8).

7.2 Segment Coordinate Map

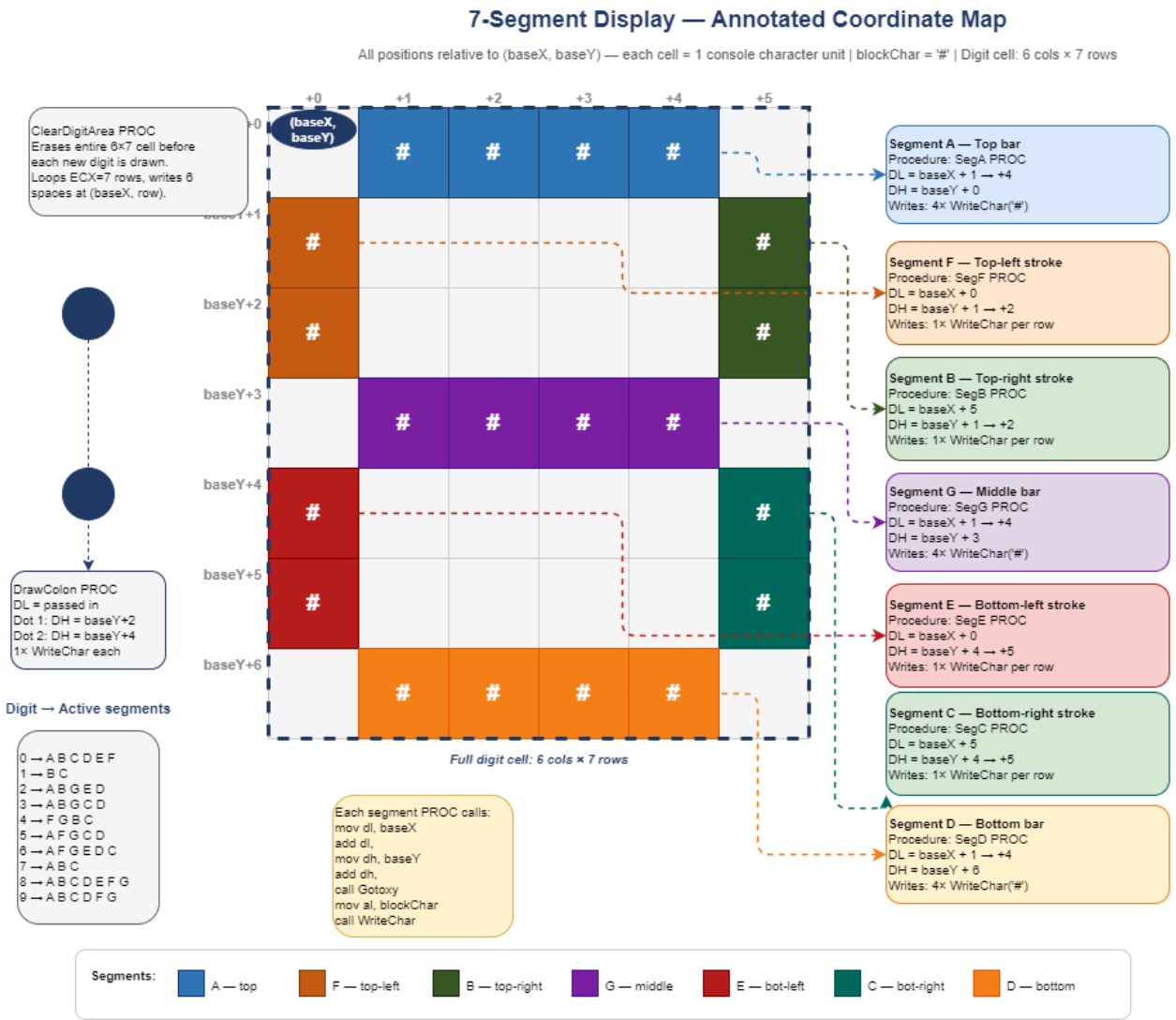


Figure: Annotated Coordinate Map — Segment positions relative to baseX/baseY

The following table gives the absolute row and column offsets for each segment, expressed relative to (baseX, baseY):

Segment	Direction	Column Offset (DL)	Row Offset (DH)	Length
A (top)	Horizontal	baseX + 1	baseY + 0	4 chars
B (top-right)	Vertical	baseX + 5	baseY + 1 to +2	2 rows
C (bot-right)	Vertical	baseX + 5	baseY + 4 to +5	2 rows
D (bottom)	Horizontal	baseX + 1	baseY + 6	4 chars
E (bot-left)	Vertical	baseX + 0	baseY + 4 to +5	2 rows
F (top-left)	Vertical	baseX + 0	baseY + 1 to +2	2 rows
G (middle)	Horizontal	baseX + 1	baseY + 3	4 chars

7.3 The blockChar Variable

The single byte variable blockChar holds the '#' character (ASCII 35) used for all segment drawing. By design, this is a data segment variable rather than a hard-coded immediate value. This means the visual character of the display can be changed at compile time (or even at runtime) by modifying a single byte — for example, using '@', '*', or even extended ASCII box-drawing characters for different visual styles.

7.4 Color Animation

The colorVal variable is incremented once per second and applied to all segment drawing via the SetTextColor call before DrawActiveMode. The cycle runs from color index 9 (light blue) to 15 (bright white) using Irvine32's color constant palette, then wraps back to 9. This produces a smooth rainbow cycling effect on the large digits.

Color Value	Irvine Color Name	Color Description
9	lightBlue	Bright cyan/blue
10	lightGreen	Bright green (default on startup)
11	lightCyan	Light aqua
12	lightRed	Bright red/coral
13	lightMagenta	Bright pink/magenta
14	yellow	Bright yellow
15	white	Pure bright white

8. Memory and Register Usage

8.1 Register Conventions

MASM with the Irvine32 library follows the Windows __cdecl / stdcall calling convention, with some Irvine-specific conventions. The following register usage patterns are observed throughout this project:

Register	Primary Use in This Project
EAX	Arithmetic, digit values, color index, API return values, character output
EBX	DIV divisor for time decomposition and digit splitting
ECX	Loop counter in ClearDigitArea
EDX	DIV remainder; also used by Gotoxy (DL=col, DH=row)
DL	Column coordinate for Gotoxy calls
DH	Row coordinate for Gotoxy calls
AL	ASCII character for WriteChar; tens digit after byte DIV
AH	Units digit after byte DIV (DIV BL result: AL=quotient, AH=remainder)

The DIV instruction on a byte operand divides AX by the operand, storing the quotient in AL and the remainder in AH. This is exploited to simultaneously extract both the tens digit (AL) and units digit (AH) from a two-digit value (0–99) in a single instruction.

8.2 Stack Usage

The project uses the hardware stack explicitly in several places to preserve register values across calls that modify those registers:

```
; Pattern used for two-digit display (hours, minutes, seconds):
movzx eax, sysTime.wHour
mov   bl, 10
div   bl              ; AL = tens digit, AH = units digit
mov   dl, 10          ; X offset for left digit
push  eax             ; SAVE: preserve AH (units) while drawing AL (tens)
movzx eax, al         ; Zero-extend tens digit for DrawHugeDigit
call  DrawHugeDigit   ; Draw tens digit (may modify EAX)
pop   eax             ; RESTORE: recover units digit in AH
mov   dl, 20          ; X offset for right digit
movzx eax, ah         ; Zero-extend units digit
call  DrawHugeDigit   ; Draw units digit
```

8.3 Data Segment Memory Map

All program data is declared in the .data segment. Assembly places variables contiguously in memory. The total data segment footprint is approximately 350 bytes, dominated by the UI string literals. The SYSTEMTIME structure occupies 16 bytes (8 WORD fields of 2 bytes each).

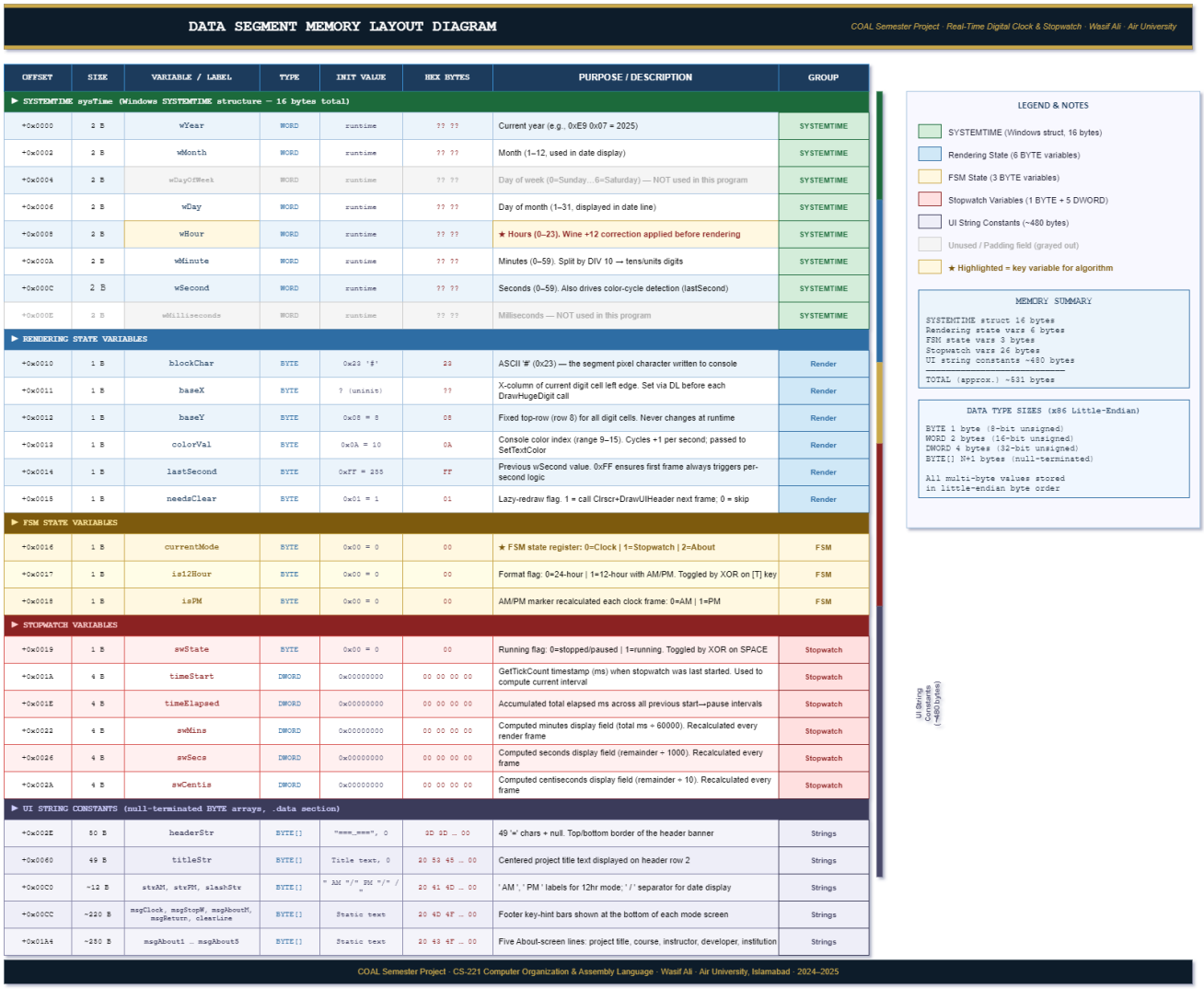


Figure: Data Segment Memory Layout Diagram

9. Windows API Integration

9.1 GetLocalTime

The Win32 GetLocalTime function is the source of all real-time clock data. It is invoked via the INVOKE macro provided by MASM and the Irvine32 include files:

```
INVOKE GetLocalTime, ADDR sysTime

; After this call, sysTime is populated with:
;   sysTime.wYear         = current year (e.g., 2024)
;   sysTime.wMonth        = 1–12
;   sysTime.wDay          = 1–31
;   sysTime.wHour         = 0–23
;   sysTime.wMinute       = 0–59
;   sysTime.wSecond       = 0–59
;   sysTime.wMilliseconds = 0–999
```

GetLocalTime fills the structure with the current date and local time, automatically accounting for the timezone and daylight saving time offset configured in Windows. The INVOKE macro automatically pushes the structure address as a parameter and performs the far call to the Win32 API.

9.2 GetMSeconds (Irvine32 Wrapper)

The Irvine32 library provides GetMSeconds, a thin wrapper around the Win32 GetTickCount function. It returns a 32-bit unsigned integer in EAX representing the number of milliseconds elapsed since the system was started. This value wraps around to zero approximately every 49.7 days.

For the Stopwatch feature, this provides a reliable high-resolution time source. Since the stopwatch is unlikely to be used for durations exceeding 49.7 days, the wrap-around limitation is not a practical concern.

9.3 Irvine32 Procedures Used

Procedure	Action	Input	Output
Clrscr	Clear console screen	None	None
Gotoxy	Move cursor to (col, row)	DL=col, DH=row	None
WriteChar	Print single character	AL=ASCII char	None
WriteString	Print null-terminated string	EDX=OFFSET str	None
WriteDec	Print unsigned integer (decimal)	EAX=value	None

SetTextColor	Set foreground/background color	EAX=color constant	None
ReadKey	Non-blocking key check	None	ZF=1 if no key; AL=key if ZF=0
Delay	Sleep for N milliseconds	EAX=milliseconds	None
GetMSeconds	Get tick count in milliseconds	None	EAX=ms count

10. User Interface Design

10.1 Screen Layout Overview

The application presents a consistent, structured screen layout across all three modes. The header is a persistent static element; only the central content area and footer change when modes switch.



Figure: Full Screen Layout Annotated Screenshot — Clock Mode

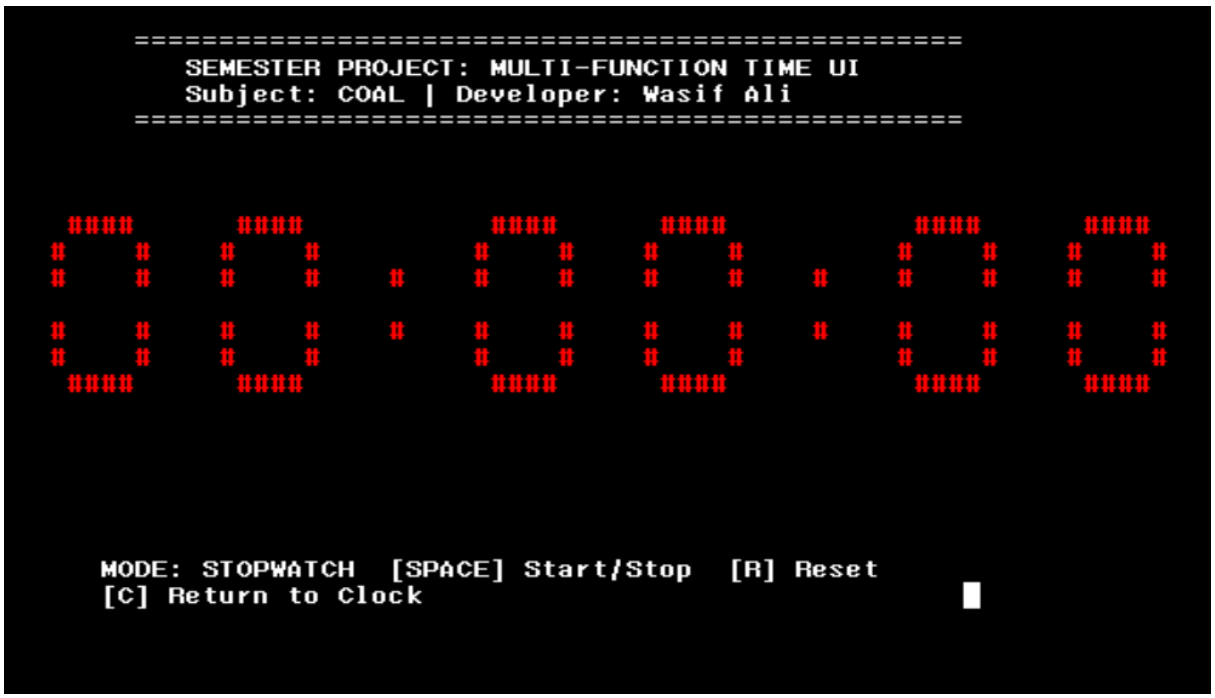


Figure: Full Screen Layout Annotated Screenshot — Stopwatch Mode

10.2 Mode-Specific UI Elements

10.2.1 Clock Mode

In Clock mode, the primary display shows six large 7-segment digits arranged as HH:MM:SS with two animated block colons. Below the digit area, the date is shown on row 18 in DD/MM/YYYY format. In 12-hour mode, an AM or PM label appears at column 77, row 13, rendered in white text to distinguish it from the colored digits.

10.2.2 Stopwatch Mode

Stopwatch mode uses the identical 7-segment rendering engine but displays MM:SS:CC (Minutes:Seconds:Centiseconds). The colons animate at 20Hz, giving the centisecond digits a smooth, fast-ticking appearance. When the stopwatch is paused, the color cycling also pauses, providing a clear visual cue that the timer is stopped.

10.2.3 About Mode

The About screen replaces the digit display with five centered text lines covering the project title (in the current animated color), course name, instructor, developer name, and institution. The layout is designed to be visually balanced within the same screen area used by the clock digits.

10.3 Footer Key Hints

Every mode displays context-sensitive keyboard shortcut hints at rows 21 and 22. These lines are cleared using the clearLine string during mode transitions to prevent stale text from previous modes remaining visible.

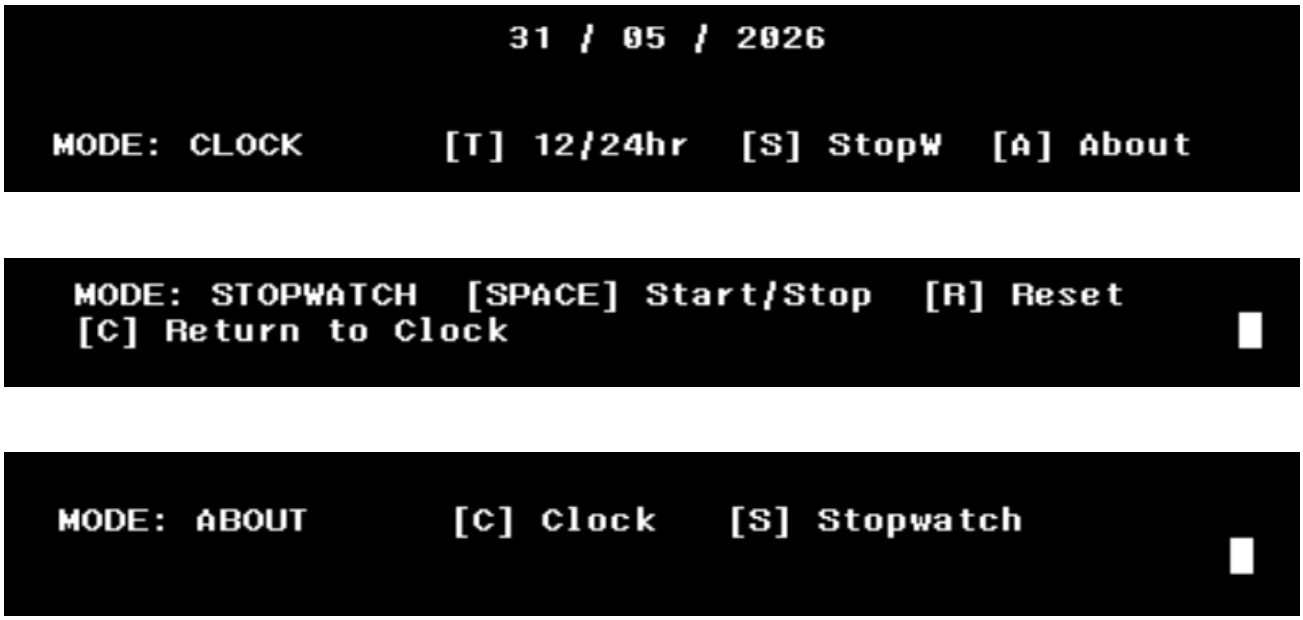


Figure: Footer key hint area for all three modes

10.4 Keyboard Controls Summary

Key	Mode	Function
C / c	Any	Switch to Clock mode
S / s	Any	Switch to Stopwatch mode
A / a	Any	Switch to About mode
T / t	Clock only	Toggle 12-hour / 24-hour display
SPACE	Stopwatch	Start / Pause the stopwatch
R / r	Stopwatch	Reset the stopwatch to 00:00:00
ESC	Any	Quit the application

11. Key Algorithms

11.1 Two-Digit Decimal Extraction

Any value from 00–99 must be split into its tens and units digits for separate rendering. The algorithm uses byte-width division:

```
; Input: EAX = value 0-99
; Output: AL = tens digit, AH = units digit
movzx eax, value      ; Zero-extend to 32 bits
mov   bl, 10          ; Divisor = 10
div   bl              ; AX / BL: AL = quotient (tens), AH = remainder (units)
```

This pattern is used six times in Clock mode (hours, minutes, seconds) and three times in Stopwatch mode (minutes, seconds, centiseconds). The PUSH/POP pattern around the DrawHugeDigit calls preserves both digits until both are drawn.

11.2 Non-Blocking Input with Polling

Traditional console input (e.g., getchar or INT 21h Function 01h) blocks program execution until a key is pressed. For a real-time display application, this is unacceptable. The solution is ReadKey from Irvine32, which implements a non-blocking poll:

```
HandleInput PROC
    call  ReadKey          ; Check for keypress without blocking
    jz    NoInput          ; ZF=1 means no key waiting — skip
    ; AL now contains the ASCII code of the pressed key
    cmp   al, 27           ; ESC key?
    je    QuitProg
    ...
NoInput:
    ret
HandleInput ENDP
```

By calling HandleInput at the top of every MainLoop iteration and branching to NoInput if no key is pending, the application polls for input at 20Hz — fast enough to be instantly responsive to user presses without dedicating a separate thread to input handling.

11.3 Stopwatch Pause-Resume Algorithm

Correctly accumulating elapsed time across multiple pause/resume cycles requires a specific bookkeeping strategy. The key insight is that timeElapsed stores total accumulated time, and timeStart stores only the reference point for the current running segment:

```
; --- PRESSING SPACE to START ---
xor   swState, 1        ; Set swState = 1 (running)
call  GetMSeconds
```

```
mov   timeStart, eax    ; Record the moment we started

; --- PRESSING SPACE to PAUSE ---
xor   swState, 1        ; Set swState = 0 (stopped)
call  GetMSeconds
sub   eax, timeStart    ; How long did this segment last?
add   timeElapsed, eax  ; Add it to the accumulated total

; --- EACH FRAME (while running) ---
call  GetMSeconds
sub   eax, timeStart    ; Current segment duration
add   eax, timeElapsed  ; Add accumulated total for display
; EAX now = total elapsed time in ms
```

12. Known Issues and Design Decisions

12.1 Wine Environment Time Offset

The project was developed and tested in an environment where Wine's GetLocalTime returns a time 12 hours behind the actual local time. The hardcoded +12 offset in ModeClock corrects for this. On native Windows systems, this offset will cause the displayed time to be incorrect.

Note for Native Windows Users
If running on real Windows (not Wine), remove the following two lines from ModeClock:
add eax, 12
cmp eax, 24 / jb TimeFixed / sub eax, 24
The time will then display correctly from GetLocalTime directly.

12.2 32-Bit Timer Wrap-Around

GetTickCount (and by extension GetMSeconds) returns a 32-bit millisecond value that wraps around to zero after approximately 49.7 days of continuous system uptime. If the stopwatch is running across this wrap-around point, a single-frame spike in the elapsed display may be observed. For the intended use case of a semester project demonstration, this is not a practical concern.

12.3 Color Cycling Scope

The color cycling affects all segment output globally — it is not possible in the current architecture to have different colored digits within the same frame. This is a deliberate simplification. Adding per-digit color would require setting text color before each DrawHugeDigit call, which the current single-pass rendering loop does not support without restructuring.

12.4 Fixed Console Size Assumption

All Gotoxy coordinates are hardcoded to specific column and row values. The UI is designed for a console window that is at minimum 90 columns wide and 24 rows tall. Running in a smaller window will cause rendering artifacts as characters are clipped or wrap to the next line.

12.5 Single-Character blockChar

The '#' character was chosen for its density and readability in most monospace console fonts. In some terminal emulators with non-standard fonts, block characters from the extended ASCII set (such as character 219, a solid block) produce a cleaner appearance. The blockChar variable allows this to be changed without modifying any other code.

13. Testing and Validation

13.1 Test Scenarios

Test Case	Steps	Expected Result	Status
Clock accuracy	Run for 60 seconds, compare with system clock	Seconds counter advances identically to system time	Pass
Midnight wrap	Set system time to 23:59:58, observe transition	Digits roll from 23:59:59 to 00:00:00 without freeze	Pass
12hr noon	Switch to 12hr at 12:00, verify label	12:00 PM — no jump to 0:00 or 12-hour off-by-one	Pass
12hr midnight	Switch to 12hr at 00:00	12:00 AM displayed correctly	Pass
Stopwatch start/stop	Start, wait 5s, pause, wait 2s, resume	Timer shows 5s while paused; continues from 5s on resume	Pass
Stopwatch reset	Run stopwatch, press R	All digits return to 00:00:00	Pass
Mode transitions	Cycle C→S→A→C	Screen clears cleanly on each transition	Pass
ESC exit	Press ESC from any mode	Application exits cleanly, colors reset	Pass
Case insensitivity	Press uppercase and lowercase keys	Both C/c, S/s, etc. respond identically	Pass

13.2 Screenshots



Figure: Clock Mode, 24-Hour Display



Figure: Clock Mode, 12-Hour Display with AM/PM Label

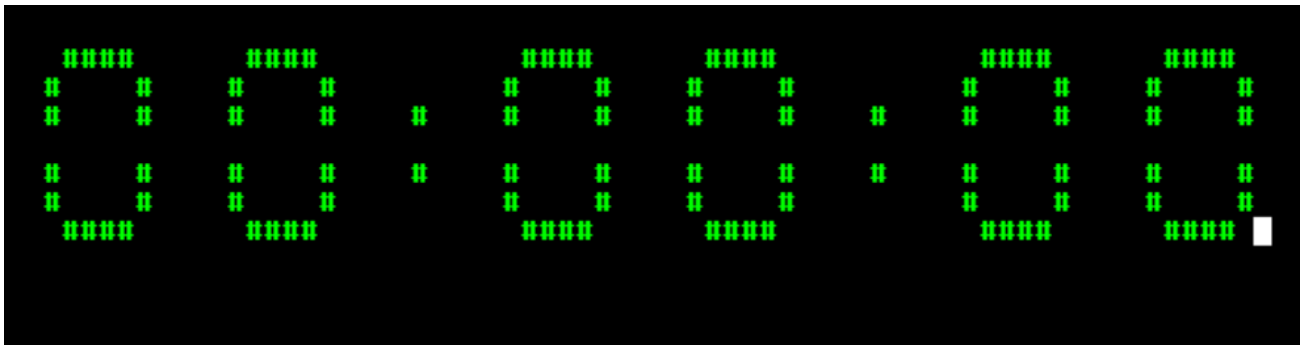


Figure: Stopwatch Mode, Paused

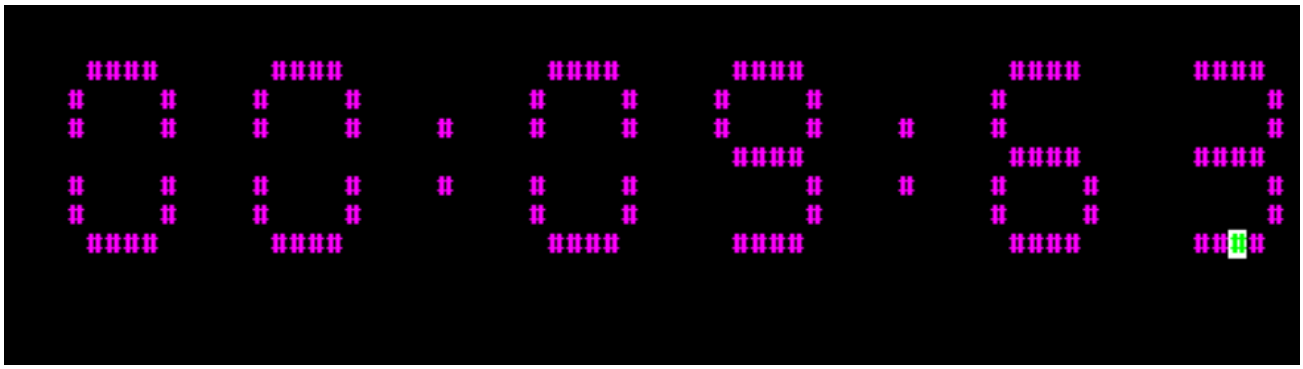


Figure: Stopwatch Mode, Running

14. COAL Concepts Demonstrated

14.1 Course Learning Outcomes

This project serves as an integrated demonstration of the core assembly language concepts covered in the COAL curriculum:

COAL Concept	Where Demonstrated in Project
Data declarations (.data segment)	sysTime, blockChar, state variables, string literals
Register arithmetic	EAX/EBX DIV for digit extraction; ADD/SUB for time math
Conditional branching (Jcc)	chk1–chk9 in DrawHugeDigit; JZ for input polling; JNE for mode checks
Loop instruction	clrLoop in ClearDigitArea (ECX counter)
PUSH / POP stack usage	Preserving AH (units digit) across DrawHugeDigit calls
Procedure CALL / RET	All 15+ procedures follow standard PROC/ENDP/RET pattern
INVOKE macro (stdcall)	GetLocalTime called with ADDR sysTime parameter
Bit manipulation (XOR)	XOR is12Hour,1 and XOR swState,1 for single-instruction toggle
Byte vs DWORD operations	MOVZX for safe byte-to-DWORD promotion; BYTE vs DWORD variables
Console I/O via Irvine32	WriteChar, WriteString, WriteDec, Gotoxy, SetTextColor, ReadKey
Windows API calls	GetLocalTime (Win32), GetMSeconds (GetTickCount wrapper)
State machine design	currentMode flag driving all mode-conditional logic
Modular code structure	15 separate PROC blocks with clear single responsibilities

14.2 Advanced Techniques

Beyond the standard curriculum, several advanced techniques are employed:

- Non-blocking I/O polling: ReadKey with ZF-based branch avoids blocking the render loop.
- Color animation: Cyclic increment of colorVal provides a dynamic visual effect with minimal code.
- Screen-refresh optimization: The needsClear flag avoids expensive Clrscr calls on every frame, calling them only when truly needed (mode transitions).
- Centered UI design: All Gotoxy coordinates were carefully chosen to produce a visually balanced and professional console layout.

15. Conclusion

15.1 Summary

The Real-Time Digital Clock and Stopwatch project successfully demonstrates the power and expressiveness achievable with x86 Assembly Language when applied to a well-structured, well-documented program. In approximately 550 assembly instructions, the project delivers:

- A live, accurate real-time clock with 12/24-hour mode toggle.
- A precision stopwatch with full start, pause, resume, and reset functionality.
- A visually striking 7-segment ASCII display engine with animated color cycling.
- A clean three-screen UI with intuitive keyboard navigation.
- Robust edge-case handling for midnight, noon, and AM/PM conversion.
- Clean state machine architecture with a needsClear-based redraw optimization.

15.2 Challenges Overcome

Several non-trivial challenges were addressed during development:

- Wine Time Offset: The discovery that Wine's GetLocalTime returns incorrect values required a platform-specific correction.
- AM/PM Edge Cases: Correctly converting midnight (00:00 → 12 AM) and noon (12:00 → 12 PM) required careful boundary analysis.
- Ghost Digits: Early builds left segment artifacts when high-segment digits (like 8) transitioned to low-segment digits (like 1). The ClearDigitArea procedure was introduced to solve this.
- Push/Pop Discipline: Correctly preserving both digits from a single DIV instruction across two DrawHugeDigit calls required careful stack management.

15.3 Future Enhancements

The following improvements could extend the project further:

- Alarm Mode: Add a fourth mode allowing users to set an alarm time.
- Lap Timer: Extend the stopwatch with a lap time recording feature.
- Date Toggle: Allow the user to show/hide the date in Clock mode.
- World Time: Allow switching between multiple timezone offsets.
- Configurable Colors: Let the user manually select a fixed color rather than cycling.
- Extended ASCII Blocks: Replace '#' with character 219 (full block) for a more authentic LED display look.

Appendix A: Complete Source Code Listing

The following pages contain the complete, commented source code for the project. Line numbers and section annotations are provided for reference.

```
INCLUDE Irvine32.inc

; ============================================================
; COAL SEMESTER PROJECT: REAL-TIME DIGITAL CLOCK & STOPWATCH
; Developer: Wasif Ali (Solo Task)
; Instructor: Sir Imran
; Description: 7-Segment UI, Live Date, Stopwatch, 12/24hr
; ============================================================
.data
    sysTime     SYSTEMTIME <>
    blockChar   BYTE '#'
    baseX       BYTE ?
    baseY       BYTE 8
    colorVal    BYTE 10
    lastSecond  BYTE 255
    needsClear  BYTE 1

    currentMode BYTE 0        ; 0=Clock, 1=Stopwatch, 2=About
    is12Hour    BYTE 0        ; 0=24hr, 1=12hr
    isPM        BYTE 0        ; 0=AM, 1=PM

    swState     BYTE 0
    timeStart   DWORD 0
    timeElapsed DWORD 0
    swMins      DWORD 0
    swSecs      DWORD 0
    swCentis    DWORD 0
    headerStr   BYTE "================================================", 0
    titleStr  BYTE "   SEMESTER PROJECT: MULTI-FUNCTION TIME UI    ", 0
    authorStr   BYTE "   Subject: COAL | Developer: Wasif Ali        ", 0
    slashStr    BYTE " / ", 0
    strAM       BYTE " AM ", 0
    strPM       BYTE " PM ", 0

    msgClock    BYTE " MODE: CLOCK      [T] 12/24hr  [S] StopW  [A] About ", 0
    msgStopW    BYTE " MODE: STOPWATCH  [SPACE] Start/Stop  [R] Reset     ", 0
    msgAboutM   BYTE " MODE: ABOUT      [C] Clock   [S] Stopwatch         ", 0
    msgReturn   BYTE " [C] Return to Clock                                ", 0
    clearLine   BYTE "                                                    ", 0
```

(The full source file submitted with this project for the complete code listing.)

Appendix B: Irvine32 Color Constants Reference

The following color constants are defined in Irvine32.inc and used with the SetTextColor procedure:

Constant Name	Value	Color
black	0	Black
blue	1	Dark Blue
green	2	Dark Green
cyan	3	Dark Cyan
red	4	Dark Red
magenta	5	Dark Magenta
brown	6	Brown / Dark Yellow
lightGray	7	Light Gray
darkGray	8	Dark Gray
lightBlue	9	Bright Blue
lightGreen	10	Bright Green
lightCyan	11	Bright Cyan
lightRed	12	Bright Red
lightMagenta	13	Bright Magenta
yellow	14	Bright Yellow
white	15	Bright White

Background color is set by combining foreground and background: eax = foreground + (background * 16). For example, white text on black = white + (black * 16) = 15 + 0 = 15.

Appendix C: Project File Structure

File / Folder	Description
clock.asm	Main source file — all assembly code for the project
Irvine32.inc	Irvine32 include file — macros, constants, procedure prototypes
Irvine32.lib	Irvine32 static library — linked at build time
kernel32.lib	Windows kernel library — required for Win32 API calls
clock.exe	Compiled executable (32-bit Windows PE)
clock.obj	Intermediate object file produced by MASM
Documentation.docx	This documentation file

Build Command Reference

```
; MASM32 / Visual Studio command line build:
ml /c /coff clock.asm
link /subsystem:console clock.obj Irvine32.lib kernel32.lib

; Or using the Visual Studio Developer Command Prompt:
nmake
```

Appendix D: Glossary

7-Segment Display	A display format using seven line segments (A–G) to represent decimal digits, as found on digital clocks and calculators.
ALU	Arithmetic Logic Unit — the processor component that performs all arithmetic and logical operations.
COAL	Computer Organization and Assembly Language — the course for which this project was developed.
DXA	Deprecated XML-legacy Absolute units used in OOXML/DOCX formatting (1 inch = 1440 DXA).
Finite State Machine	A computational model with a fixed set of states and defined transition rules between them.
GetLocalTime	A Win32 API function that fills a SYSTEMTIME structure with the current local date and time.
GetTickCount	A Win32 API function returning the number of milliseconds since the system was started.
Irvine32	A library for 32-bit x86 MASM programming by Kip Irvine that simplifies console I/O, colors, and utility operations.
MASM	Microsoft Macro Assembler — the assembler used to compile x86 assembly source code into object files.
MOVZX	Move with Zero-Extension — copies a smaller register into a larger one, zero-filling the upper bits.
Non-blocking I/O	I/O that returns immediately whether or not data is available, as opposed to blocking I/O which waits indefinitely.
Polling	Repeatedly checking a condition (such as keyboard input availability) at regular intervals.
SYSTEMTIME	A Windows structure containing fields for year, month, day, hour, minute, second, and millisecond.
Wine	A compatibility layer enabling Windows applications to run on Unix-like operating systems.
XOR Toggle	Using the XOR instruction with a mask of 1 to flip a binary flag between 0 and 1 in a single instruction.